```
$username = basename(realpath($_
GET['user']));
$filename = "/home/users/$username";
readfile($filename);
```

Tools of the secure trade

Dotdotpwn is a very flexible Perl-based intelligent fuzzer tool, which detects several directory-traversal vulnerabilities on HTTP/FTP servers. For Windows systems, it also detects the presence of *boot.ini* on vulnerable systems through directory-traversal vulnerabilities. It is available for free download at *http://www.darknet.org.uk/tag/dotdotpwn/*, along with its documentation.

http://yehg.net/lab/pr0js/pentest/wordlists/injections/Traversal.txt contains many path-traversal URLs that are frequently used by attackers. This domain also contains other good resources on security and ethical hacking.

Source-code disclosure

Source-code disclosure, another variant of the path-traversal attack, is a widely prevalent vulnerability in Web applications, which lets attackers extract source code and configuration files. Such vulnerabilities are found mostly in websites that offer to download files using dynamic scripts. The attacker uses this technique to obtain the source code of server-side scripts like ASP, JSP or PHP files, to discover Web application logic, including database structure, source code comments, parameters and other possibly exploitable vulnerabilities of the code. Let's understand this using a simple attack scenario.

An attack scenario

Let's assume a website uses the following PHP code, which initiates a file download from the server:

```
<?php
if(isset($_GET['file']))
{
   $file = $_GET['file'];
   readfile($file);
}
?>
```

A valid URL for the above script is *http://www.example.com/downloads.php?file=arpit.zip*, but the attacker's malicious URL could be *http://www.example.com/downloads.php?file=login.php*, which returns to the attacker the contents of the file '*login.php*'. With this, the attacker learns about the filters and checks in '*login.php*', and even the names of other crucial database and systems configuration files.

To secure your application from source-code disclosure, the application should use a pre-defined list of permissible file types, and reject any request for a different type.

Directory listing leakage

This is a commonly-found vulnerability in many Web servers. When a Web server receives a request for a directory rather than an actual file, it may respond in one of three ways:

1. It may return a (configurable) default resource within the directory, such as *index.html*, *home.html*, *default.htm*, *default.asp*, *default.aspx*, *index.php*, etc.
2. It may return an HTTP status code 403 error message, indicating that the request is not permitted.
3. It may return a listing showing the contents of the directory. This happens when default resources are not present in the directory.

In many situations, directory listings do not have any relevance to security. For example, disclosing the listing of an images directory may be completely inconsequential. Indeed, directory listings are often intentionally allowed because they provide a built-in means of navigating sites containing static content. But still, some files and directories are often, unintentionally, left within the Web root of servers, including:

- Application-generated files: Web-authoring applications often generate files that find their way to the server. A good example is a popular FTP client, WS_FTP, which places a log file into each folder it transfers to the Web server. Since people often transfer folders in bulk, the log files themselves are transferred, exposing file paths and allowing the attacker to enumerate all files. Another example is CityDesk, which places a list of all files in the root folder of the site, in a file named *citydesk.xml*.
- Configuration-management files: Configuration-management tools create many files with metadata. Again, these files are frequently transferred to the website. CVS, the most popular configuration-management tool, keeps its files in a special folder named CVS. This folder is created as a sub-folder of every user-created folder, and it contains the files *Entries*, *Repository*, and *Root*.
- Backup files: Text editors often create backup files with extensions such as *~*, *.bak*, *.old*, *.bkp*, and *.swp*. When changes are performed directly on the server, backup files remain there. Even when created on a development server or workstation, by virtue of a bulk folder FTP transfer, they end up on the production server.
- Exposed application files: Script-based applications often consist of files not meant to be accessed directly, but instead used as libraries or subroutines. Exposure happens if these files' extensions are not recognised by the Web server as a script. Instead of executing the script, the server sends the full source code in response to a request. With access to the source code, the attacker can look for security-related bugs. Also, these files can sometimes be manipulated to circumvent application logic.
- Web server's crucial information: This is often displayed at the end of the listing page, and contains the server name, version number and other important information. Such information can be used to launch specific exploits against the Web server.